INTERFACES GRÁFICAS DE ESCRITORIO EN JAVA

Swing y la arquitectura Modelo - Vista - Controlador

MATERIAL DE CLASE · DESARROLLO DE SISTEMAS ORIENTADO A OBJETOS

Esteban Santini · 2026

AGENDA DE LA CLASE

01 · UN POCO DE HISTORIA

Las primeras interfaces gráficas en Java: applets, AWT y el camino hacia Swing. Otras alternativas actuales como JavaFX.

02 · SWING Y SUS COMPONENTES

La jerarquía de clases de Swing y el trabajo con JFrame, JLabel, JTextField y JButton: instanciar, configurar y componer.

03 · LAYOUTS Y EVENTOS

Los layout managers básicos (FlowLayout, BorderLayout, GridLayout) y el manejo de eventos con ActionListener.

04 · ARQUITECTURA MVC

Separar la aplicación en capas: Modelo, Vista y Controlador. Ejemplo completo de una aplicación Swing con MVC.

Un poco de historia

Java nació en 1995 con la promesa de “escribir una vez, ejecutar en cualquier lado”. Las interfaces gráficas acompañaron esa promesa desde el primer día, con varias generaciones de tecnología.

1995

Applets y AWT

Java 1.0 llega con applets para el navegador y AWT como toolkit gráfico.

1998

Swing (Java 1.2)

Parte de las Java Foundation Classes: componentes livianos y flexibles.

2008

JavaFX

Nueva plataforma para interfaces ricas, hoy proyecto OpenJFX.

2017+

Fin de los applets

Deprecados en Java 9 y eliminados definitivamente en Java 17.

APPLETS: JAVA EN EL NAVEGADOR

La primera apuesta gráfica de Java

- Pequeñas aplicaciones Java que se ejecutaban embebidas dentro de una página web, mediante la etiqueta <applet> y un plugin del navegador.
- Corrían dentro de un “sandbox” de seguridad que limitaba el acceso al disco y a la red de la máquina del usuario.
- Tenían un ciclo de vida propio con los métodos init(), start(), stop() y destroy(), invocados por el navegador.
- Fueron muy populares a fines de los 90 pero cayeron en desuso: problemas de seguridad, plugins pesados y el avance de JavaScript/HTML5.

REGISTRO HISTÓRICO

Java 9 (2017)
La API de applets se marca como deprecada.

Java 17 (2021)
Se eliminan definitivamente del JDK.

Hoy no deben usarse en ningún desarrollo nuevo.

AWT: ABSTRACT WINDOW TOOLKIT

El primer toolkit gráfico del JDK (1995)

AWT delega cada componente en un “peer” nativo del sistema operativo: un botón AWT en Windows es un botón real de Windows. Por eso se dice que sus componentes son “pesados” (heavyweight).

A FAVOR

- Aspecto 100 % nativo en cada sistema operativo.
- Rápido: el dibujado lo hace el propio SO.
- Sigue siendo la base de Swing: ventanas, eventos y layouts provienen de AWT (paquete java.awt).

EN CONTRA

- Solo ofrece el “mínimo común denominador”: los componentes que existen en todos los SO.
- La misma pantalla podía verse distinta en cada plataforma.
- Componentes poco personalizables y catálogo muy limitado (sin tablas, árboles, pestañas…).

LLEGA SWING (1998)

Java Foundation Classes — paquete javax.swing

- Componentes livianos (lightweight): Java los dibuja píxel a píxel, sin depender de los controles nativos del sistema operativo.
- Mismo aspecto y comportamiento en todas las plataformas, con Look & Feel intercambiable (Metal, Nimbus, aspecto del SO…).
- Catálogo mucho más rico: tablas (JTable), árboles (JTree), pestañas (JTabbedPane), diálogos (JOptionPane) y muchos más.
- Construido sobre AWT: reutiliza sus ventanas, su sistema de eventos y sus layout managers.
- Sigue incluido en el JDK actual: no requiere instalar nada extra y hay miles de sistemas de escritorio que lo usan en producción.

CONVENCIÓN

Los componentes Swing empiezan con “J”:

```
Button   →  JButton
Label    →  JLabel
Frame    →  JFrame

AWT: java.awt.*
Swing: javax.swing.*
```

OTRAS HERRAMIENTAS DE ESCRITORIO

El ecosistema más allá de Swing

JAVAFX / OPENJFX

Sucesor moderno propuesto por Sun/Oracle (2008). Estilos con CSS, interfaces declaradas en FXML, animaciones y multimedia.

Desde Java 11 se distribuye separado del JDK, como el proyecto OpenJFX.

SWT

Standard Widget Toolkit, creado por IBM para Eclipse. Usa componentes nativos del sistema operativo, como AWT, pero con un catálogo completo.

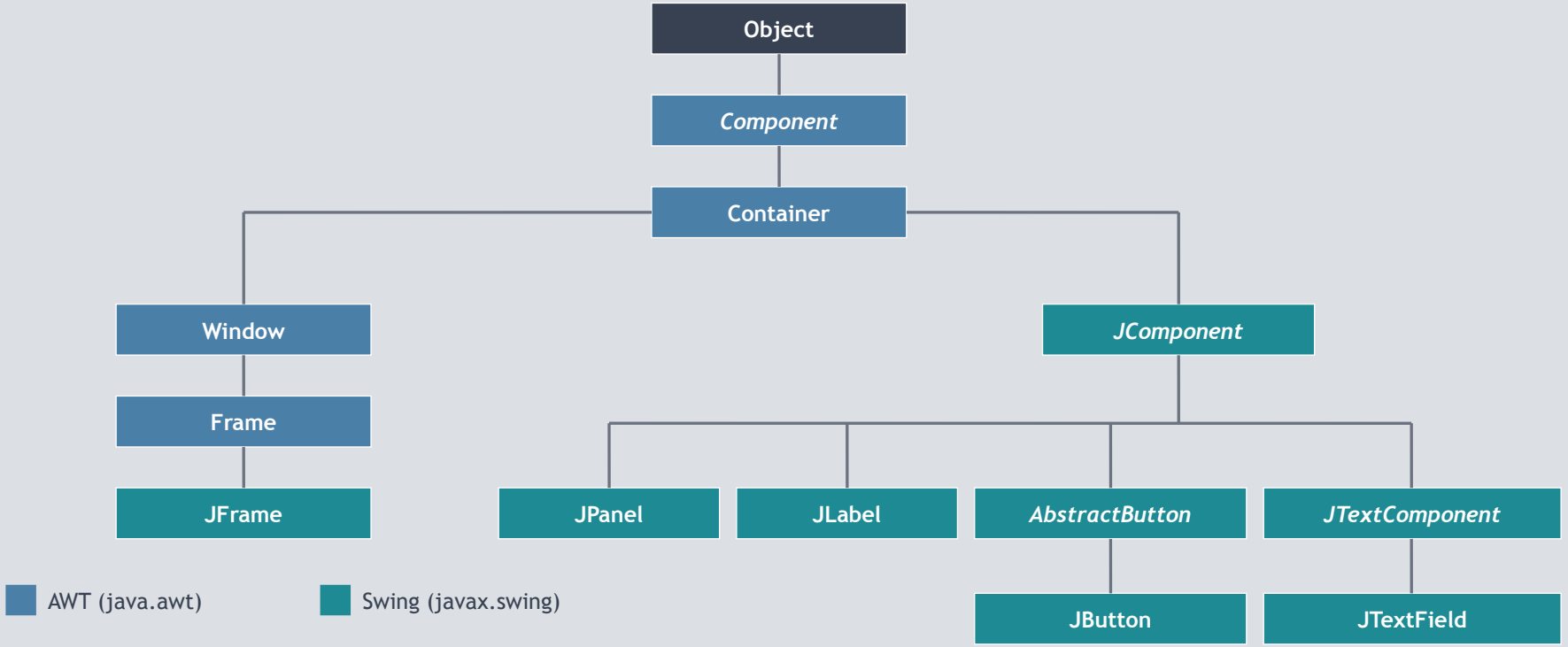
Es la base del IDE Eclipse.

Y HOY…

Swing sigue siendo el estándar de facto para escritorio en Java y la opción más simple para aprender.

Alternativas actuales: Jetpack Compose Desktop (Kotlin), Electron o aplicaciones web.

JERARQUÍA DE CLASES DE SWING



En cursiva: clases abstractas. Todo componente Swing hereda de Component y Container: por eso conserva los métodos de AWT.

CÓMO SE TRABAJA CON SWING

Cuatro pasos que se repiten siempre

1

INSTANCIAR

Crear los objetos de las clases de componentes con new: JFrame, JLabel, JTextField, JButton…

2

CONFIGURAR

Setear los atributos con métodos set: título, tamaño, texto, fuente, colores, estado…

3

COMPONER

Agregar cada componente a su contenedor con add(), organizados por un layout manager.

4

ESCUCHAR

Registrar los eventos con addActionListener() y compañía para reaccionar al usuario.

La interfaz se arma "por composición": contenedores que contienen componentes.

JFRAME: LA VENTANA PRINCIPAL

- Es la ventana de nivel superior: toda aplicación Swing de escritorio tiene al menos un JFrame.
- Aporta el marco del sistema: barra de título, botones de minimizar / maximizar / cerrar.
- setDefaultCloseOperation define qué pasa al cerrar: EXIT_ON_CLOSE termina la aplicación.
- setLocationRelativeTo(null) centra la ventana en la pantalla.
- Recién con setVisible(true) la ventana aparece: conviene llamarlo al final, con todo ya compuesto.

```
// Instanciar
JFrame ventana = new JFrame();

// Configurar atributos
ventana.setTitle("Mi primera ventana");
ventana.setSize(400, 300);
ventana.setLocationRelativeTo(null);
ventana.setDefaultCloseOperation(
        JFrame.EXIT_ON_CLOSE);
ventana.setLayout(new FlowLayout());

// Mostrar (siempre al final)
ventana.setVisible(true);
```

JLABEL: ETIQUETAS DE TEXTO

- Muestra texto (o una imagen) que el usuario no puede editar.
- Se usa para titular campos, mostrar mensajes o resultados.
- setText() / getText() permiten cambiar y leer su contenido en tiempo de ejecución.
- Atributos típicos: setFont(), setForeground() (color del texto), setHorizontalAlignment().
- Acepta HTML sencillo: new JLabel("<html>Hola</html>").

```
// Instanciar con el texto inicial
JLabel etiqueta = new JLabel("Nombre:");

// Configurar atributos
etiqueta.setFont(
    new Font("Arial", Font.BOLD, 14));
etiqueta.setForeground(Color.BLUE);

// Componer: agregarla a la ventana
ventana.add(etiqueta);

// Más adelante, cambiar el texto
etiqueta.setText("¡Hola, mundo!");
```

JTEXTFIELD: ENTRADA DE TEXTO

- Campo de una sola línea para que el usuario escriba texto.
- El constructor recibe el ancho en columnas: new JTextField(20).
- getText() devuelve lo que escribió el usuario: es la forma básica de leer datos de la interfaz.
- setText() carga un valor por programa; setEditable(false) lo vuelve de solo lectura.
- Para varias líneas existe JTextArea; para contraseñas, JPasswordField.

```
// Instanciar (20 columnas de ancho)
JTextField campoNombre =
        new JTextField(20);

// Configurar atributos
campoNombre.setText("Ana");
campoNombre.setEditable(true);

// Componer
ventana.add(campoNombre);

// Leer lo que escribió el usuario
String nombre = campoNombre.getText();
```

JBUTTON: BOTONES DE ACCIÓN

- Botón que el usuario presiona para disparar una acción.
- El texto se define en el constructor o con setText().
- setEnabled(false) lo deshabilita (se ve gris y no responde).
- setToolTipText() muestra una ayuda al pasar el mouse.
- Lo importante ocurre con addActionListener(): registrar quién responde cuando se hace clic (próxima diapositiva).

```
// Instanciar
JButton boton = new JButton("Saludar");

// Configurar atributos
boton.setEnabled(true);
boton.setToolTipText(
    "Hacé clic para saludar");

// Componer
ventana.add(boton);

// Falta lo más importante:
// ¿qué pasa cuando lo presionan?
```

EVENTOS: RESPONDER AL USUARIO

Swing usa el patrón Observer: el componente avisa a sus “escuchas” (listeners) cuando ocurre algo. Para un clic de botón se registra un ActionListener y se implementa actionPerformed().

```
// Opción 1: clase anónima
boton.addActionListener(
  new ActionListener() {
    @Override
    public void actionPerformed(
            ActionEvent e) {
      etiqueta.setText("¡Hola!");
    }
});
```

```
// Opción 2: lambda (Java 8+)
boton.addActionListener(
    e -> etiqueta.setText("¡Hola!"));

// Requiere importar:
// java.awt.event.ActionListener
// java.awt.event.ActionEvent
```

Ambas formas son equivalentes: la lambda es más corta porque ActionListener tiene un solo método.

LAYOUT MANAGERS BÁSICOS

En Swing no se posiciona con coordenadas fijas: cada contenedor delega la ubicación y el tamaño de sus componentes en un layout manager, que reacomoda todo cuando la ventana cambia de tamaño.

FLOWLAYOUT

Ubica los componentes en fila, de izquierda a derecha, y salta de línea cuando no hay lugar.

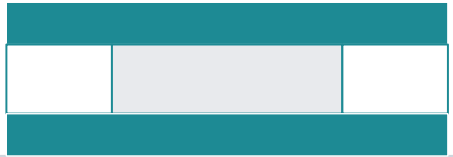
Es el layout por defecto de JPanel.



BORDERLAYOUT

Divide el contenedor en cinco zonas: NORTH, SOUTH, EAST, WEST y CENTER.

Es el layout por defecto de JFrame.



GRIDLAYOUT

Grilla de filas × columnas donde todas las celdas miden lo mismo.

new GridLayout(2, 3) crea 2 filas y 3 columnas.



EJEMPLO COMPLETO: COMPONER LA INTERFAZ

```
import javax.swing.*;
import java.awt.*;

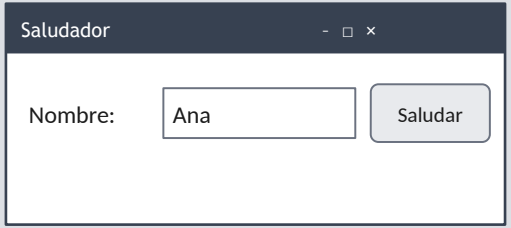
public class AppSaludo {
    public static void main(String[] args) {
        JFrame ventana = new JFrame("Saludador");          // 1. instanciar
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ventana.setLayout(new FlowLayout());               // 2. configurar

        JLabel etiqueta = new JLabel("Nombre:");
        JTextField campo = new JTextField(15);
        JButton boton = new JButton("Saludar");

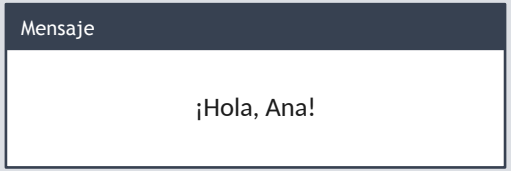
        boton.addActionListener(e ->                       // 4. escuchar
            JOptionPane.showMessageDialog(ventana,
                "¡Hola, " + campo.getText() + "!"));

        ventana.add(etiqueta);                             // 3. componer
        ventana.add(campo);
        ventana.add(boton);
        ventana.setSize(350, 120);
        ventana.setVisible(true);
    }
}
```

RESULTADO



Al hacer clic:



Funciona, pero todo el código queda en el main. ¿Se puede organizar mejor?

CADA VENTANA, UNA CLASE: EXTENDER JFRAME

- En vez de armar la ventana en el main, se crea una clase propia que hereda de JFrame.
- Los componentes pasan a ser atributos de la clase: accesibles desde cualquier método.
- El constructor configura la ventana y llama a un método que inicializa y compone los componentes.
- Como heredamos, llamamos directamente a setTitle(), add(), setSize()… sin escribir “ventana.”.
- El main queda de una sola línea: crear la ventana y mostrarla.

```
public class VentanaPrincipal extends JFrame {

    private JLabel etiqueta;
    private JTextField campo;
    private JButton boton;

    public VentanaPrincipal() {
        super("Saludador");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        inicializarComponentes();
        setSize(350, 120);
    }

    private void inicializarComponentes() {
        etiqueta = new JLabel("Nombre:");
        campo = new JTextField(15);
        boton = new JButton("Saludar");
        add(etiqueta); add(campo); add(boton);
    }
}
// main:  new VentanaPrincipal().setVisible(true);
```

PANELES REUTILIZABLES: EXTENDER JPANEL

- JPanel es un contenedor intermedio: agrupa componentes relacionados dentro de una ventana.
- Extender JPanel permite crear “piezas” de interfaz reutilizables en distintas ventanas.
- Cada panel maneja su propio layout, independiente del layout de la ventana.
- Exponemos getters para que otras clases accedan a lo que necesitan (el texto ingresado, el botón…).
- La ventana se arma combinando paneles: por ejemplo, uno en CENTER y otro en SOUTH.

```
public class PanelSaludo extends JPanel {

    private JTextField campo = new JTextField(15);
    private JButton boton = new JButton("Saludar");

    public PanelSaludo() {
        setLayout(new FlowLayout());
        add(new JLabel("Nombre:"));
        add(campo);
        add(boton);
    }

    public String getNombre() {
        return campo.getText();
    }

    public JButton getBoton() {
        return boton;
    }
}

// En la ventana:
// add(new PanelSaludo(), BorderLayout.CENTER);
```

DOS ESTILOS DE CONSTRUCCIÓN

COMPOSICIÓN EN EL MAIN

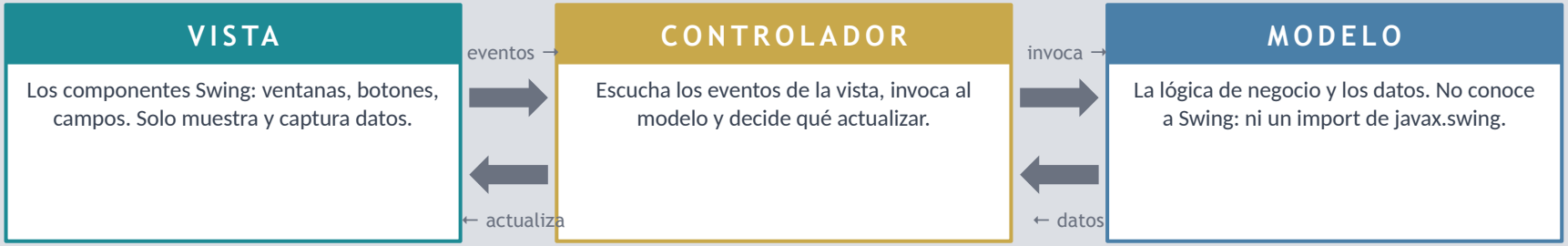
- Se instancia JFrame directamente y se arma todo en un solo método.
- Ideal para ejemplos chicos, prototipos y para entender los fundamentos.
- Problema: a medida que la interfaz crece, el main se vuelve inmanejable.
- Los componentes son variables locales: nadie más puede consultarlos.

HERENCIA: CLASES PROPIAS

- Cada ventana o panel es una clase que extiende JFrame o JPanel.
- Los componentes son atributos: estado accesible desde toda la clase.
- Favorece la reutilización, la lectura y el trabajo en equipo.
- Es el estilo que usan los diseñadores visuales de los IDE y el que usaremos en la materia.

ARQUITECTURA EN CAPAS: MVC

Modelo–Vista–Controlador separa la aplicación en tres capas con responsabilidades claras. La interfaz puede cambiar sin tocar la lógica de negocio, y la lógica se puede probar sin abrir ninguna ventana.



- Regla de oro: el modelo nunca importa javax.swing; la vista nunca contiene reglas de negocio.
- El controlador es el único que conoce a ambos: recibe los eventos y coordina.
- Vamos a construir el “Saludador” completo con estas tres clases en las próximas diapositivas.

MVC EN SWING — 1) EL MODELO

- Una clase común y corriente: atributos y métodos con la lógica de negocio.
- No tiene ni un solo import de javax.swing ni de java.awt.
- Acá viven las reglas: cómo se genera el saludo y cuántos saludos se hicieron.
- Al no depender de la interfaz, se puede probar con JUnit o reutilizar en una app web.

```
// MODELO: lógica de negocio, sin Swing
public class SaludadorModelo {

    private int cantidadSaludos = 0;

    public String generarSaludo(String nombre) {
        cantidadSaludos++;
        if (nombre == null || nombre.isBlank()) {
            return "¡Hola, anónimo!";
        }
        return "¡Hola, " + nombre + "! (saludo N.° "
               + cantidadSaludos + ")";
    }

    public int getCantidadSaludos() {
        return cantidadSaludos;
    }
}
```

MVC EN SWING — 2) LA VISTA

- Extiende JFrame: es una ventana con sus componentes como atributos.
- Solo construye y compone la interfaz: no decide qué se responde al usuario.
- Expone getters y métodos de actualización para que el controlador la maneje.
- Si mañana cambiamos JLabel por un diálogo, el modelo ni se entera.

```
// VISTA: solo interfaz, sin lógica de negocio
public class SaludadorVista extends JFrame {

    private JTextField campoNombre = new JTextField(12);
    private JButton botonSaludar = new JButton("Saludar");
    private JLabel resultado = new JLabel("---");

    public SaludadorVista() {
        super("Saludador MVC");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        add(new JLabel("Nombre:"));
        add(campoNombre);
        add(botonSaludar);
        add(resultado);
        setSize(420, 130);
    }

    public String getNombre() { return campoNombre.getText(); }
    public JButton getBotonSaludar() { return botonSaludar; }
    public void mostrarSaludo(String s) { resultado.setText(s); }
}
```

MVC EN SWING — 3) EL CONTROLADOR

```
// CONTROLADOR: conecta la vista con el modelo
public class SaludadorControlador {

    private SaludadorModelo modelo;
    private SaludadorVista vista;

    public SaludadorControlador(SaludadorModelo m,
                                SaludadorVista v) {
        this.modelo = m;
        this.vista = v;
        vista.getBotonSaludar()
             .addActionListener(e -> saludar());
    }

    private void saludar() {
        String s = modelo.generarSaludo(vista.getNombre());
        vista.mostrarSaludo(s);
    }
}
```

```
public static void main(String[] args) {
    SaludadorVista vista = new SaludadorVista();
    new SaludadorControlador(new SaludadorModelo(), vista);
    vista.setVisible(true);
}
```

- Registra los listeners sobre los componentes de la vista.
- Ante cada evento: lee la vista, invoca al modelo y actualiza la vista.
- El main solo “cablea” las tres piezas y muestra la ventana.
- **Tres archivos, tres responsabilidades: eso es la arquitectura en capas.**

VARIANTE: ACCIONES COMO OBJETOS

- La Action API representa cada operación de la aplicación como un objeto: texto, tooltip, atajo de teclado, estado enabled y el código a ejecutar, todo junto.
- Se crea extendiendo AbstractAction e implementando actionPerformed().
- Las propiedades visuales se cargan con putValue() y sus claves (SHORT_DESCRIPTION, MNEMONIC_KEY…).
- Un mismo objeto Action puede asociarse a la vez a un botón, un ítem de menú y una toolbar.
- El modelo no cambia en nada: esta variante solo toca la capa de control y el enlace con la vista.

```
// ACCIÓN: una operación como objeto
public class SaludarAction extends AbstractAction {

    private final SaludadorModelo modelo;
    private final SaludadorVista vista;

    public SaludarAction(SaludadorModelo modelo,
                         SaludadorVista vista) {
        super("Saludar");              // texto
        putValue(SHORT_DESCRIPTION,
                 "Genera un saludo");  // tooltip
        putValue(MNEMONIC_KEY,
                 KeyEvent.VK_S);       // Alt+S
        this.modelo = modelo;
        this.vista = vista;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        vista.mostrarSaludo(
            modelo.generarSaludo(vista.getNombre()));
    }
}
```

ACTION API — LA VISTA SE ENLAZA

- El botón y el ítem de menú se crean vacíos: texto, tooltip y mnemonic los toman de la acción al hacer setAction().
- La vista ya no expone getBotonSaludar(): mejora el encapsulamiento.
- El punto de enlace setAccionSaludar() decide internamente qué componentes representan la acción.
- Si el controlador hace accion.setEnabled(false), botón e ítem de menú se deshabilitan juntos.

```
public class SaludadorVista extends JFrame {
    private JTextField campoNombre = new JTextField(12);
    private JButton botonSaludar = new JButton();     // sin texto
    private JMenuItem itemSaludar = new JMenuItem();  // sin texto
    private JLabel resultado = new JLabel("---");

    public SaludadorVista() {
        super("Saludador MVC");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        JMenu menu = new JMenu("Acciones");
        menu.add(itemSaludar);
        JMenuBar barra = new JMenuBar();
        barra.add(menu);  setJMenuBar(barra);
        add(new JLabel("Nombre:"));
        add(campoNombre); add(botonSaludar); add(resultado);
        setSize(420, 160);
    }

    // La vista decide quiénes representan la acción
    public void setAccionSaludar(Action a) {
        botonSaludar.setAction(a);
        itemSaludar.setAction(a);
    }

    public String getNombre() { return campoNombre.getText(); }
    public void mostrarSaludo(String s) { resultado.setText(s); }
}
```

ACTION API — CONTROLADOR Y ARRANQUE

```
// CONTROLADOR: crea las acciones y las enlaza
public class SaludadorControlador {

    public SaludadorControlador(SaludadorModelo modelo,
                                SaludadorVista vista) {
        vista.setAccionSaludar(
            new SaludarAction(modelo, vista));
        // más operaciones, más acciones:
        // vista.setAccionLimpiar(new LimpiarAction(...));
    }
}
```

```
public class App {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            SaludadorModelo modelo = new SaludadorModelo();
            SaludadorVista vista = new SaludadorVista();
            new SaludadorControlador(modelo, vista);
            vista.setVisible(true);
        });
    }
}
```

- El controlador queda reducido a crear y “cablear” acciones: ese catálogo de acciones es la capa controladora.
- SwingUtilities.invokeLater() crea la interfaz en el hilo de eventos (EDT): la buena práctica profesional.
- **Con una sola acción parece burocracia; con veinte operaciones, es orden.**

ACTION API: A FAVOR Y EN CONTRA

A FAVOR

- Una operación, un objeto: texto, tooltip, atajo y lógica viven juntos, sin duplicación.
- La misma acción sirve para botón, menú y toolbar; setEnabled() los sincroniza a todos.
- Escala muy bien: los IDE (NetBeans, IntelliJ) organizan toda su funcionalidad como acciones.
- Mejora el encapsulamiento: la vista deja de exponer sus componentes internos.

EN CONTRA

- Más clases y más código para casos simples: para un botón único e interno, la lambda es más directa.
- actionPerformed() sigue siendo público: cualquier clase puede dispararlo.
- setAction() pisa el texto y tooltip configurados en el componente: hay que definir quién manda.
- putValue() y sus claves son poco intuitivas al principio.

Criterio práctico: lambdas para acciones simples y locales; Action API cuando una operación aparece en varios lugares o cambia de estado según el contexto.

RESUMEN Y BUENAS PRÁCTICAS

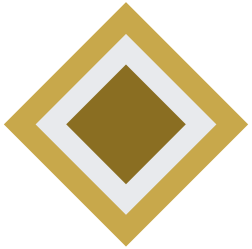
- Applets y AWT son historia: hoy, para escritorio en Java, se aprende y se trabaja con Swing (y JavaFX como alternativa).
- El ciclo de trabajo es siempre el mismo: instanciar → configurar → componer → escuchar eventos.
- Usá layout managers, nunca posiciones fijas: la interfaz se adapta al tamaño de la ventana.
- Cada ventana o panel en su propia clase, extendiendo JFrame o JPanel.
- Separá en capas con MVC: modelo sin Swing, vista sin lógica, controlador coordinando.
- Detalle profesional: la interfaz debe crearse en el hilo de eventos con SwingUtilities.invokeLater(() -> …).

SIDE QUEST

Práctica: construir una calculadora con Swing aplicando MVC.

Vista con GridLayout, modelo con las operaciones y controlador con los eventos.

Documentación: docs.oracle.com → Java Tutorials → Creating a GUI with Swing



MUCHAS GRACIAS

“Las interfaces cambian; los buenos diseños permanecen.”

Preguntas y consultas: ahora, o por los canales de la materia.
En la próxima clase: práctica de Swing + MVC.